

Video Classification

Edge Intelligence

Ashish Nikam

25MML0015

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions
```

Load pretrained model

```
model = MobileNetV2(weights='imagenet')
```

```
def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)
```

```
    if not cap.isOpened():
        raise ValueError("Cannot open video")
```

```
    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

```
    step = max(1, total_frames // max_frames) if total_frames > 0 else 1
```

```
    for i in range(total_frames):
```

```
        ret, frame = cap.read()
```

```
        if not ret:
            break
```

```
        if i % step == 0:
```

```
            frame = cv2.resize(frame, (224, 224))
```

```
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

```
            frames.append(frame)
```

```
        if len(frames) >= max_frames:
            break
```

```
    cap.release()
```

```
    return np.array(frames)
```

```

def classify_video(video_path):
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    frames = preprocess_input(frames)
    predictions = model.predict(frames, verbose=0)

    avg_pred = np.mean(predictions, axis=0)
    top_preds = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

    print("\nTop Predictions:")
    for label in top_preds[0]:
        print(f"{label[1]} : {label[2]*100:.2f}%")

    print("\nFinal Prediction:", top_preds[0][0][1])

```

```

video_path = "/content/drive/MyDrive/026c7465-309f6d33.mp4"
classify_video(video_path)

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224.h5
14536120/14536120 ————— 2s 0us/step

Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 ————— 0s 0us/step

Top Predictions:
cab : 17.92%
traffic_light : 6.14%
streetcar : 5.89%
jinrikisha : 3.63%
limousine : 3.56%

Final Prediction: cab

```

# =====
# 1 Mount Google Drive
# =====
from google.colab import drive
drive.mount('/content/drive')

# =====
# 2 Install & Import Libraries
# =====
!pip install -q tensorflow opencv-python matplotlib

```

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

```

```

import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions

# =====
# 3 Load Pretrained Model
# =====
print("Loading MobileNetV2 model...")
model = MobileNetV2(weights='imagenet')
print("Model loaded successfully!\n")

# =====
# 4 Frame Extraction Function
# =====
def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    step = max(1, total_frames // max_frames)

    count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)

        count += 1
        if len(frames) >= max_frames:
            break

    cap.release()
    return np.array(frames)

video_path = "/content/drive/MyDrive/026c7465-309f6d33.mp4"

# =====
# 6 Extract Frames
# =====
extracted_frames = extract_frames(video_path)

if len(extracted_frames) == 0:
    raise ValueError("No frames extracted. Check video path.")

print("Extracted Frames Shape:", extracted_frames.shape)
print("Data Type:", extracted_frames.dtype)

```

```

# =====
# 7 Display First 5 Frames
# =====
plt.figure(figsize=(15,5))
for i in range(min(5, len(extracted_frames))):
    plt.subplot(1,5,i+1)
    plt.imshow(extracted_frames[i])
    plt.title(f"Frame {i+1}")
    plt.axis("off")
plt.show()

# =====
# 8 Show Pixel Values (5x5 Region)
# =====
print("\nTop-left 5x5 Pixel Region of First Frame:")
print(extracted_frames[0, 0:5, 0:5, :])

# =====
# 9 Normalize (0-255 → 0-1)
# =====
normalized_frames = extracted_frames / 255.0

print("\nNormalized Range:")
print("Min:", normalized_frames.min())
print("Max:", normalized_frames.max())

# =====
# 10 Model Preprocessing
# =====
processed_frames = preprocess_input(extracted_frames)

# =====
# 1 1 Predict on Frames
# =====
print("\nRunning predictions...")
predictions = model.predict(processed_frames, verbose=0)

# Average predictions across frames
avg_prediction = np.mean(predictions, axis=0)

# Decode top 5 predictions
top_predictions = decode_predictions(np.expand_dims(avg_prediction, axis=0), top=5)

print("\nTop 5 Predictions:")
for pred in top_predictions[0]:
    print(f"{pred[1]} : {pred[2]*100:.2f}%")

print("\nFinal Prediction:", top_predictions[0][0][1])

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Loading MobileNetV2 model...

Model loaded successfully!

Extracted Frames Shape: (20, 224, 224, 3)

Data Type: uint8



Top-left 5x5 Pixel Region of First Frame:

```
[[[ 60 67 53]
 [ 57 63 50]
 [125 131 119]
 [103 108 101]
 [ 99 103 98]]]
```

```
[[ 55 61 48]
 [ 60 66 53]
 [ 83 89 77]
 [ 88 93 87]
 [121 126 120]]]
```

```
[[ 48 54 41]
 [ 62 67 55]
 [ 68 74 63]
 [ 80 85 79]
 [103 108 102]]]
```

```
[[ 45 51 38]
 [ 50 56 43]
 [ 60 66 54]
 [ 81 86 80]
 [ 76 81 75]]]
```

```
[[ 43 49 36]
 [ 44 47 35]
 [ 50 52 41]
 [102 107 101]
 [ 74 78 73]]]
```

Normalized Range:

Min: 0.0

Max: 1.0

Running predictions...

Top 5 Predictions:

cab : 17.92%

traffic_light : 6.14%

streetcar : 5.89%

jinrikisha : 3.63%

limousine : 3.56%

Final Prediction: cab

Display all extracted frames in one long horizontal strip

```
num_frames = len(extracted_frames)
```

```
plt.figure(figsize=(40, 4)) # Wide figure
```

```
for i in range(num_frames):
    plt.subplot(1, num_frames, i + 1)
    plt.imshow(extracted_frames[i])
    plt.title(f"Frame {i+1}", fontsize=8)
    plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



```
# =====
# ADVANCED VIDEO ANALYSIS
# =====
```

```
import matplotlib.pyplot as plt
from collections import Counter
```

```
# -----
# 1 RGB Channel Analysis
# -----
```

```
def analyze_rgb(frames):
    avg_r, avg_g, avg_b = [], [], []

    for frame in frames:
        avg_r.append(np.mean(frame[:, :, 0]))
        avg_g.append(np.mean(frame[:, :, 1]))
        avg_b.append(np.mean(frame[:, :, 2]))
```

```
    return avg_r, avg_g, avg_b
```

```
avg_r, avg_g, avg_b = analyze_rgb(extracted_frames)
```

```
# Plot RGB trends
plt.figure(figsize=(10,4))
plt.plot(avg_r, label='Red Channel')
plt.plot(avg_g, label='Green Channel')
plt.plot(avg_b, label='Blue Channel')
plt.title("RGB Channel Intensity Across Frames")
plt.xlabel("Frame Number")
plt.ylabel("Average Pixel Intensity")
plt.legend()
plt.show()
```

```
# -----
# 2 RGB Histogram (First Frame)
# -----

plt.figure(figsize=(10,4))
colors = ['r', 'g', 'b']

for i, color in enumerate(colors):
    plt.hist(extracted_frames[0][:,:,i].ravel(), bins=50, color=color, alpha=0.5)

plt.title("RGB Histogram (First Frame)")
plt.show()
```

```
# -----
# 3 Frame-wise Classification
# -----

processed_frames = preprocess_input(extracted_frames)
predictions = model.predict(processed_frames, verbose=0)

frame_labels = []
frame_confidences = []

for i, pred in enumerate(predictions):
    decoded = decode_predictions(np.expand_dims(pred, axis=0), top=1)
    label = decoded[0][0][1]
    confidence = decoded[0][0][2]

    frame_labels.append(label)
    frame_confidences.append(confidence)

    print(f"Frame {i+1}: {label} ({confidence*100:.2f}%)")
```

```
# -----
# 4 Confidence Graph
# -----

plt.figure(figsize=(10,4))
plt.plot(frame_confidences, marker='o')
plt.title("Prediction Confidence per Frame")
plt.xlabel("Frame Number")
plt.ylabel("Confidence")
plt.show()
```

```
# -----
# 5 Majority Voting
# -----

most_common_label = Counter(frame_labels).most_common(1)[0][0]

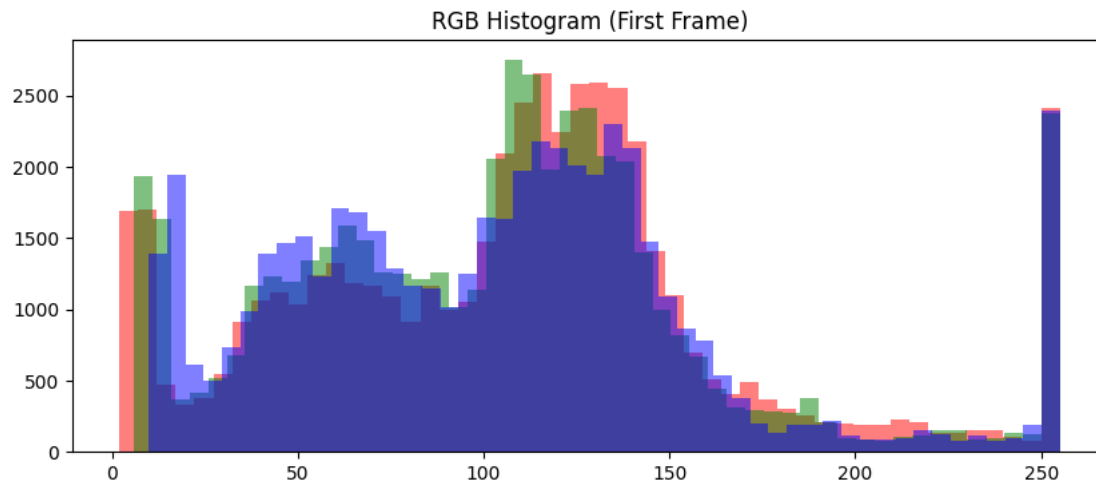
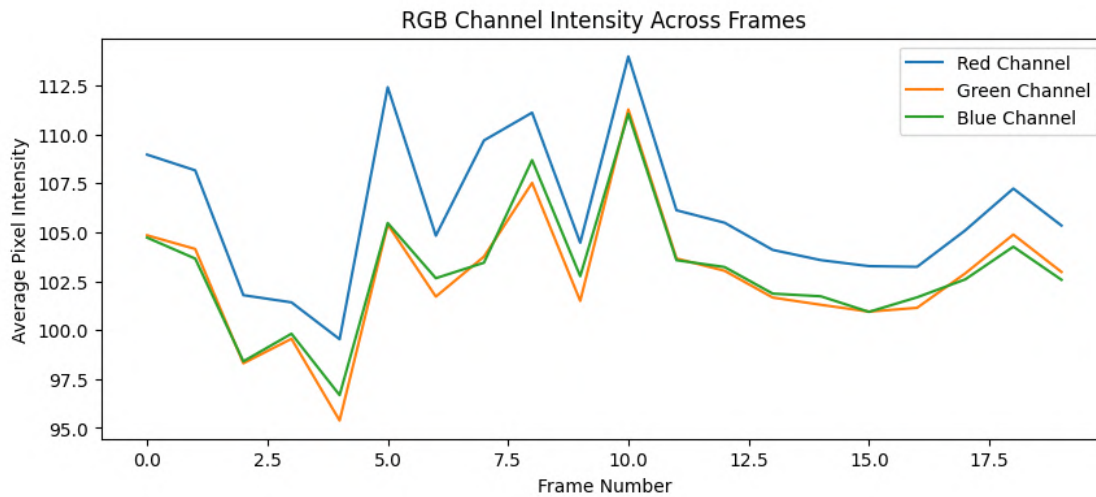
print("\nMajority Voting Prediction:", most_common_label)
```

```
# -----
#  Weighted Average Prediction
# -----
```

```
avg_prediction = np.mean(predictions, axis=0)
top_predictions = decode_predictions(np.expand_dims(avg_prediction, axis=0), top=5)
```

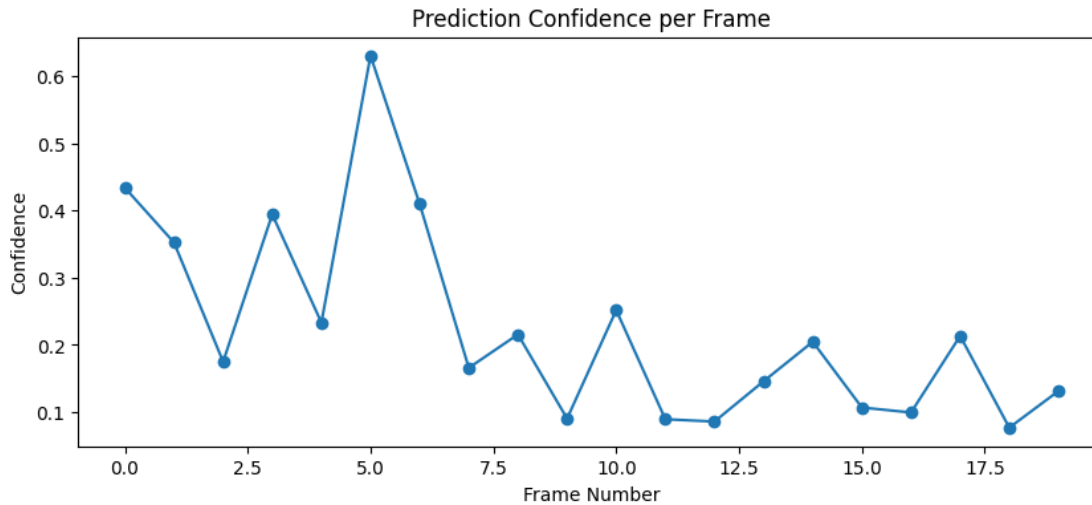
```
print("\nTop 5 Overall Predictions:")
for pred in top_predictions[0]:
    print(f"{pred[1]} : {pred[2]*100:.2f}%")
```

```
print("\nFinal Weighted Prediction:", top_predictions[0][0][1])
```



Frame 1: cab (43.38%)
 Frame 2: streetcar (35.24%)
 Frame 3: cab (17.51%)
 Frame 4: cab (39.42%)
 Frame 5: cab (23.32%)
 Frame 6: cab (63.02%)
 Frame 7: cab (40.97%)
 Frame 8: limousine (16.59%)
 Frame 9: cab (21.56%)
 Frame 10: streetcar (9.04%)

Frame 11: cab (25.17%)
Frame 12: maze (8.95%)
Frame 13: streetcar (8.60%)
Frame 14: jinrikisha (14.60%)
Frame 15: unicycle (20.47%)
Frame 16: jinrikisha (10.69%)
Frame 17: jinrikisha (9.95%)
Frame 18: cab (21.34%)
Frame 19: maze (7.70%)
Frame 20: garbage_truck (13.14%)



Majority Voting Prediction: cab

Top 5 Overall Predictions:

cab : 17.92%
traffic_light : 6.14%
streetcar : 5.89%
jinrikisha : 3.63%
limousine : 3.56%

Final Weighted Prediction: cab

```
import cv2
```

```
cap = cv2.VideoCapture("/content/drive/MyDrive/026c7465-309f6d33.mp4")
```

```
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

```
fps = cap.get(cv2.CAP_PROP_FPS)
```

```
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
```

```
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
```

```
print("Total Frames:", total_frames)
```

```
print("FPS:", fps)
```

```
print("Width:", width)
```

```
print("Height:", height)
```

```
cap.release()
```

Total Frames: 2398
FPS: 59.94005994005994
Width: 1280.0
Height: 720.0

```
# =====  
# 1 Mount Google Drive  
# =====
```

```
from google.colab import drive  
drive.mount('/content/drive')
```

```
# =====  
# 2 Install Libraries  
# =====
```

```
!pip install -q tensorflow opencv-python matplotlib
```

```
# =====  
# 3 Import Libraries  
# =====
```

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
from collections import Counter  
import tensorflow as tf  
from tensorflow.keras.applications import MobileNetV2  
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions
```

```
# =====  
# 4 Load CNN Model  
# =====
```

```
print("Loading MobileNetV2...")  
model = MobileNetV2(weights="imagenet")  
print("Model Loaded\n")
```

```
# =====  
# 5 Video Path  
# =====
```

```
video_path = "/content/drive/MyDrive/026c7465-309f6d33.mp4"
```

```
# =====  
# 6 Video Properties  
# =====
```

```
cap = cv2.VideoCapture(video_path)
```

```
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))  
fps = cap.get(cv2.CAP_PROP_FPS)  
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)  
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
```

```
print("Video Properties")
print("Total Frames:", total_frames)
print("FPS:", fps)
print("Width:", width)
print("Height:", height)
```

```
cap.release()
```

```
# =====
#  Frame Extraction
# =====
```

```
def extract_frames(video_path, max_frames=20):
```

```
    cap = cv2.VideoCapture(video_path)
```

```
    frames = []
```

```
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
```

```
    step = max(1, total_frames // max_frames)
```

```
    count = 0
```

```
    while True:
```

```
        ret, frame = cap.read()
```

```
        if not ret:
            break
```

```
        if count % step == 0:
```

```
            frame = cv2.resize(frame,(224,224))
```

```
            frame = cv2.cvtColor(frame,cv2.COLOR_BGR2RGB)
```

```
            frames.append(frame)
```

```
        count += 1
```

```
        if len(frames) >= max_frames:
            break
```

```
    cap.release()
```

```
    return np.array(frames)
```

```
extracted_frames = extract_frames(video_path)
```

```
print("\nExtracted Frames Shape:",extracted_frames.shape)
```

```
# =====
#  Display All Frames
# =====
```

```

num_frames = len(extracted_frames)

plt.figure(figsize=(40,4))

for i in range(num_frames):

    plt.subplot(1,num_frames,i+1)
    plt.imshow(extracted_frames[i])
    plt.title(f"Frame {i+1}",fontsize=8)
    plt.axis("off")

plt.show()

# =====
# 9 RGB Pixel Inspection
# =====

print("\nTop-left 5x5 RGB values of first frame\n")

print(extracted_frames[0,0:5,0:5,:])

# =====
# 10 RGB Channel Analysis
# =====

avg_r=[]
avg_g=[]
avg_b=[]

for frame in extracted_frames:

    avg_r.append(np.mean(frame[:,0]))
    avg_g.append(np.mean(frame[:,1]))
    avg_b.append(np.mean(frame[:,2]))

plt.figure(figsize=(10,4))

plt.plot(avg_r,label="Red")
plt.plot(avg_g,label="Green")
plt.plot(avg_b,label="Blue")

plt.title("RGB Intensity Across Frames")
plt.xlabel("Frame")
plt.ylabel("Pixel Intensity")

plt.legend()

plt.show()

# =====
# 11 RGB Histogram
# =====

```

```

plt.figure(figsize=(10,4))

colors=["r","g","b"]

for i,color in enumerate(colors):

    plt.hist(extracted_frames[0][:,:,i].ravel(),
             bins=50,
             color=color,
             alpha=0.5)

plt.title("RGB Histogram (Frame 1)")
plt.show()

# =====
# 1 2 Linear Intensity Transformation
# Image Negative
#  $S = (L-1) - r$ 
# =====

negative_frame = 255 - extracted_frames[0]

plt.figure(figsize=(8,4))

plt.subplot(1,2,1)
plt.title("Original")
plt.imshow(extracted_frames[0])
plt.axis("off")

plt.subplot(1,2,2)
plt.title("Negative Image")
plt.imshow(negative_frame)
plt.axis("off")

plt.show()

# =====
# 1 3 Edge Detection
# =====

gray = cv2.cvtColor(extracted_frames[0],cv2.COLOR_RGB2GRAY)

edges = cv2.Canny(gray,100,200)

plt.figure(figsize=(6,4))
plt.title("Edge Detection")
plt.imshow(edges,cmap="gray")
plt.axis("off")
plt.show()

# =====
# 1 4 Draw Shapes + Text
# =====

```

```

frame = extracted_frames[0].copy()

cv2.rectangle(frame,(40,40),(180,180),(0,255,0),3)

cv2.circle(frame,(120,120),40,(255,0,0),3)

cv2.putText(frame,
    "OpenCV Practice",
    (10,20),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (0,0,255),
    2)

plt.imshow(frame)
plt.title("Shapes + Text")
plt.axis("off")
plt.show()

```

```

# =====
# 1 5 Motion Detection
# =====

```

```

cap = cv2.VideoCapture(video_path)

ret,frame1 = cap.read()
ret,frame2 = cap.read()

motion_frames=[]

while ret:

    diff = cv2.absdiff(frame1,frame2)

    gray = cv2.cvtColor(diff,cv2.COLOR_BGR2GRAY)

    blur = cv2.GaussianBlur(gray,(5,5),0)

    _,thresh = cv2.threshold(blur,20,255,cv2.THRESH_BINARY)

    motion_frames.append(thresh)

    frame1 = frame2

    ret,frame2 = cap.read()

    if len(motion_frames)>10:
        break

cap.release()

plt.imshow(motion_frames[0],cmap="gray")
plt.title("Motion Detection")
plt.axis("off")
plt.show()

```

```
# =====
# 1 6 Deep Learning Classification
# =====
```

```
processed = preprocess_input(extracted_frames)
```

```
predictions = model.predict(processed,verbose=0)
```

```
frame_labels=[]
```

```
frame_conf=[]
```

```
print("\nFrame-wise Predictions\n")
```

```
for i,p in enumerate(predictions):
```

```
    decoded = decode_predictions(np.expand_dims(p,axis=0),top=1)
```

```
    label = decoded[0][0][1]
```

```
    conf = decoded[0][0][2]
```

```
    frame_labels.append(label)
```

```
    frame_conf.append(conf)
```

```
    print(f'Frame {i+1}: {label} ({conf*100:.2f}%)')
```

```
# =====
# 1 7 Confidence Graph
# =====
```

```
plt.figure(figsize=(10,4))
```

```
plt.plot(frame_conf,marker="o")
```

```
plt.title("Prediction Confidence Across Frames")
```

```
plt.xlabel("Frame")
```

```
plt.ylabel("Confidence")
```

```
plt.show()
```

```
# =====
# 1 8 Majority Voting
# =====
```

```
final_label = Counter(frame_labels).most_common(1)[0][0]
```

```
print("\nMajority Voting Prediction:",final_label)
```

```
# =====
# 1 9 Weighted Prediction
# =====
```

```

avg_prediction = np.mean(predictions,axis=0)

top_preds = decode_predictions(np.expand_dims(avg_prediction,axis=0),top=5)

print("\nTop Predictions:")

for p in top_preds[0]:

    print(p[1],":",round(p[2]*100,2),"%")

```

```

print("\nFinal Prediction:",top_preds[0][0][1])

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Loading MobileNetV2...

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224.h5

14536120/14536120 ————— 2s 0us/step

Model Loaded

Video Properties

Total Frames: 2398

FPS: 59.94005994005994

Width: 1280.0

Height: 720.0

Extracted Frames Shape: (20, 224, 224, 3)



Top-left 5x5 RGB values of first frame

```

[[[ 60  67  53]
   [ 57  63  50]
   [125 131 119]
   [103 108 101]
   [ 99 103  98]]

```

```

[[ 55  61  48]
   [ 60  66  53]
   [ 83  89  77]
   [ 88  93  87]
   [121 126 120]]

```

```

[[ 48  54  41]
   [ 62  67  55]
   [ 68  74  63]
   [ 80  85  79]
   [103 108 102]]

```

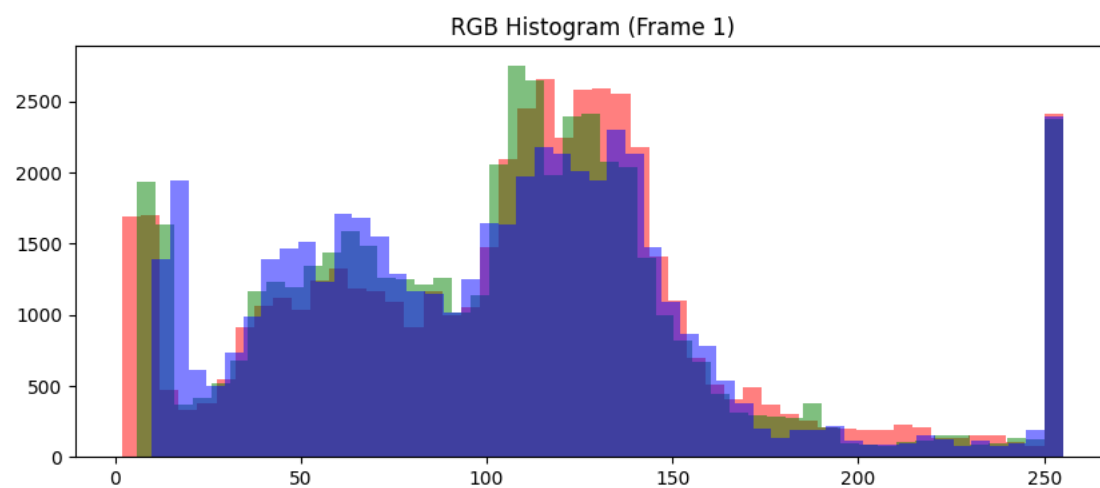
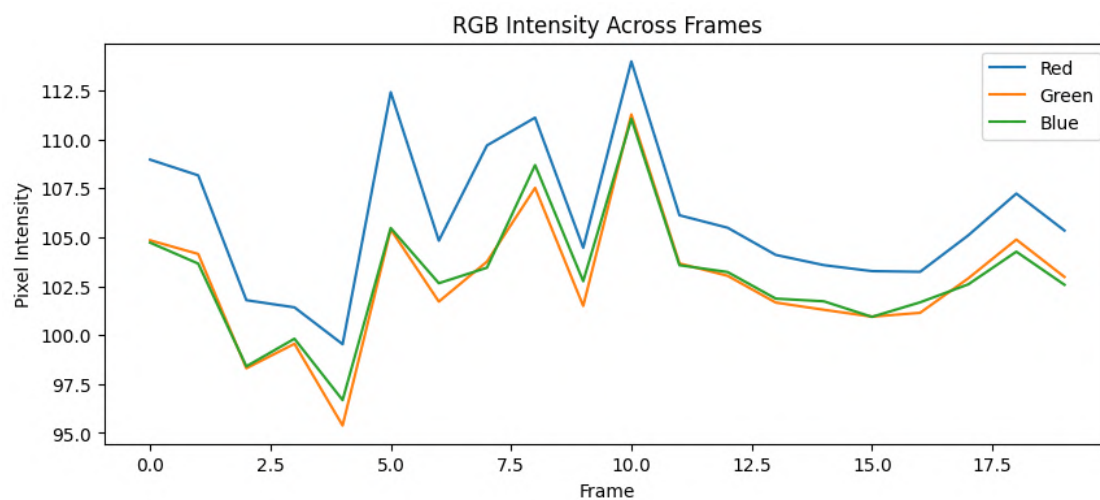
```

[[ 45  51  38]
   [ 50  56  43]
   [ 60  66  54]
   [ 81  86  80]]

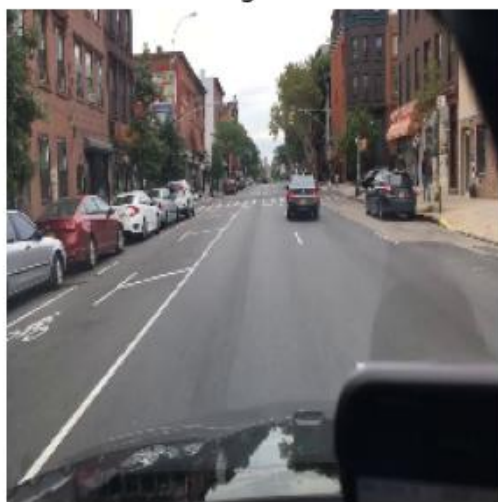
```

[76 81 75]]

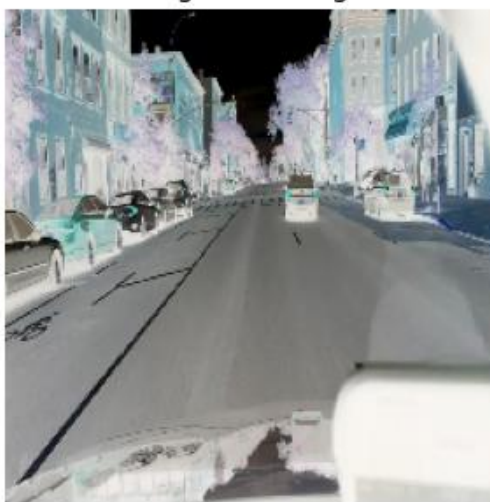
[[43 49 36]
[44 47 35]
[50 52 41]
[102 107 101]
[74 78 73]]]



Original



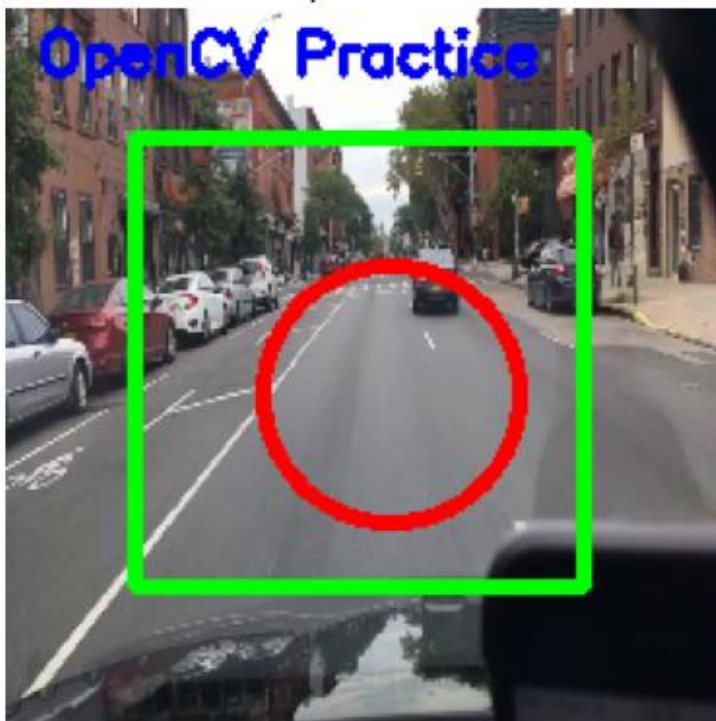
Negative Image



Edge Detection



Shapes + Text



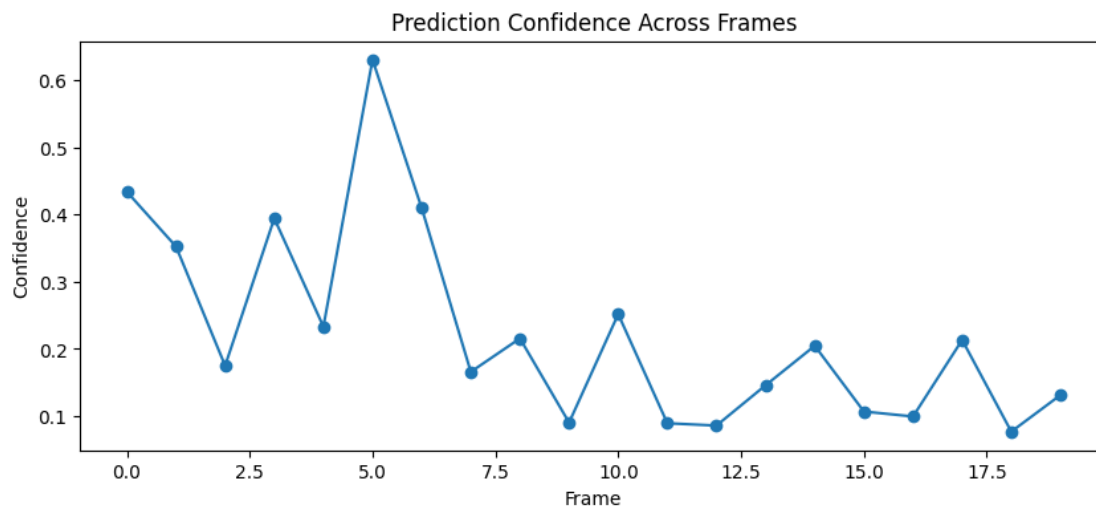
Motion Detection



Frame-wise Predictions

Downloading data from https://storage.googleapis.com/download.tensorflow.org/data/imagenet_class_index.json
35363/35363 ————— 0s 1us/step

Frame 1: cab (43.38%)
Frame 2: streetcar (35.24%)
Frame 3: cab (17.51%)
Frame 4: cab (39.42%)
Frame 5: cab (23.32%)
Frame 6: cab (63.02%)
Frame 7: cab (40.97%)
Frame 8: limousine (16.59%)
Frame 9: cab (21.56%)
Frame 10: streetcar (9.04%)
Frame 11: cab (25.17%)
Frame 12: maze (8.95%)
Frame 13: streetcar (8.60%)
Frame 14: jinrikisha (14.60%)
Frame 15: unicycle (20.47%)
Frame 16: jinrikisha (10.69%)
Frame 17: jinrikisha (9.95%)
Frame 18: cab (21.34%)
Frame 19: maze (7.70%)
Frame 20: garbage_truck (13.14%)



Majority Voting Prediction: cab

Top Predictions:

cab : 17.92 %

traffic_light : 6.14 %

streetcar : 5.89 %

jinrikisha : 3.63 %

limousine : 3.56 %

Final Prediction: cab

```
import cv2
```

```
# Open laptop webcam
```

```
cap = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
# Convert to grayscale
```

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
# Canny edge detection
```

```
edges = cv2.Canny(gray, 100, 200)
```

```
# Show original video
```

```
cv2.imshow("Original Camera", frame)
```

```
# Show edges
```

```
cv2.imshow("Canny Edge Detection", edges)
```

```
# Press q to exit
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

```
    break
```

```
cap.release()
cv2.destroyAllWindows()
```

```
-----
error                                Traceback (most recent call last)
/tmp/ipykernel_403/3678544209.py in <cell line: 0>()
    28
    29 cap.release()
---> 30 cv2.destroyAllWindows()
```

error: OpenCV(4.13.0) /io/opencv/modules/highgui/src/window.cpp:1295: error: (-2:Unspecified error) The function is not implemented. Rebuild the library with Windows, GTK+ 2.x or Cocoa support. If you are on Ubuntu or Debian, install libgtk2.0-dev and pkg-config, then re-run cmake or configure script in function 'cvDestroyAllWindows'

```
# =====
# Install and Import Libraries
# =====
```

```
!pip install opencv-python-headless
```

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from IPython.display import display, Javascript
from google.colab.output import eval_js
from base64 import b64decode
```

```
# =====
# Webcam Capture Function
# =====
```

```
def take_photo(filename='photo.jpg', quality=0.8):
```

```
    js = Javascript("""
        async function takePhoto(quality) {

            const div = document.createElement('div');
            const capture = document.createElement('button');
            capture.textContent = 'Capture Image';
            div.appendChild(capture);

            const video = document.createElement('video');
            video.style.display = 'block';

            const stream = await navigator.mediaDevices.getUserMedia({video: true});

            document.body.appendChild(div);
            div.appendChild(video);

            video.srcObject = stream;
            await video.play();

            google.colab.output.setIframeHeight(document.documentElement.scrollHeight, true);
```

```

    await new Promise((resolve) => capture.onclick = resolve);

    const canvas = document.createElement('canvas');
    canvas.width = video.videoWidth;
    canvas.height = video.videoHeight;

    canvas.getContext('2d').drawImage(video, 0, 0);

    stream.getVideoTracks()[0].stop();
    div.remove();

    return canvas.toDataURL('image/jpeg', quality);
  }
  ""
)

```

display(js)

```

data = eval_js('takePhoto({}).format(quality)')
binary = b64decode(data.split(',')[1])

```

```

with open(filename, 'wb') as f:
    f.write(binary)

```

```

return filename

```

```

# =====
# Capture Webcam Image
# =====

```

```

print("Opening webcam...")

```

```

photo = take_photo()

```

```

# =====
# Read Captured Image
# =====

```

```

frame = cv2.imread(photo)

```

```

print("Original Image")
cv2_imshow(frame)

```

```

# =====
# Convert to Grayscale
# =====

```

```

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

```

```

print("Grayscale Image")
cv2_imshow(gray)

```

```

# =====
# Apply Canny Edge Detection
# =====

```

```
edges = cv2.Canny(gray, 100, 200)
```

```
print("Canny Edge Detection Result")
```

```
cv2_imshow(edges)
```

```
# =====
```

```
# Display Pixel Information
```

```
# =====
```

```
print("\nImage Shape:", frame.shape)
```

```
print("Pixel Intensity Range:", frame.min(), "to", frame.max())
```

Requirement already satisfied: opencv-python-headless in /usr/local/lib/python3.12/dist-packages (4.13.0.92)

Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python-headless) (2.0.2)

Opening webcam...

<IPython.core.display.Javascript object>

```
-----  
MessageError Traceback (most recent call last)
```

```
/tmp/ipykernel_403/568617620.py in <cell line: 0>()
```

```
    71 print("Opening webcam...")
```

```
    72
```

```
---> 73 photo = take_photo()
```

```
    74
```

```
    75 # =====
```

```
/tmp/ipykernel_403/568617620.py in take_photo(filename, quality)
```

```
    56 display(js)
```

```
    57
```

```
---> 58 data = eval_js('takePhoto({})'.format(quality))
```

```
    59 binary = b64decode(data.split(',')[1])
```

```
    60
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/output/_js.py in eval_js(script, ignore_result, timeout_sec)
```

```
    38 if ignore_result:
```

```
    39     return
```

```
---> 40 return _message.read_reply_from_input(request_id, timeout_sec)
```

```
    41
```

```
    42
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/_message.py in read_reply_from_input(message_id, timeout_sec)
```

```
    101     ):
```

```
    102     if 'error' in reply:
```

```
--> 103         raise MessageError(reply['error'])
```

```
    104     return reply.get('data', None)
```

```
    105
```

MessageError: NotAllowedError: Permission denied

```
# =====
```

```
# Import libraries
```

```
# =====
```

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow
from IPython.display import display, Javascript
from google.colab.output import eval_js
from base64 import b64decode
```

```
# =====
# Webcam capture function
# =====
```

```
def take_photo(filename='photo.jpg', quality=0.8):
```

```
    js = Javascript("""
        async function takePhoto(quality) {

            const video = document.createElement('video');
            const div = document.createElement('div');
            const button = document.createElement('button');

            button.textContent = 'Capture';
            div.appendChild(button);
            div.appendChild(video);

            document.body.appendChild(div);

            const stream = await navigator.mediaDevices.getUserMedia({video: true});
            video.srcObject = stream;
            await video.play();

            await new Promise(resolve => button.onclick = resolve);

            const canvas = document.createElement('canvas');
            canvas.width = video.videoWidth;
            canvas.height = video.videoHeight;

            canvas.getContext('2d').drawImage(video,0,0);

            stream.getTracks().forEach(track => track.stop());
            div.remove();

            return canvas.toDataURL('image/jpeg',quality);
        }
    """)
```

```
    display(js)
```

```
    data = eval_js('takePhoto({})'.format(quality))
```

```
    binary = b64decode(data.split(',')[1])
```

```
    with open(filename, 'wb') as f:
        f.write(binary)
```

```
    return filename
```

```

# =====
# Capture image from webcam
# =====

print("Opening webcam... Please allow camera access.")

photo = take_photo()

# =====
# Read image
# =====

frame = cv2.imread(photo)

print("Original Image")
cv2_imshow(frame)

# =====
# Convert to grayscale
# =====

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

print("Grayscale Image")
cv2_imshow(gray)

# =====
# Apply Canny Edge Detection
# =====

edges = cv2.Canny(gray,100,200)

print("Canny Edge Detection Result")
cv2_imshow(edges)

# =====
# Display pixel information
# =====

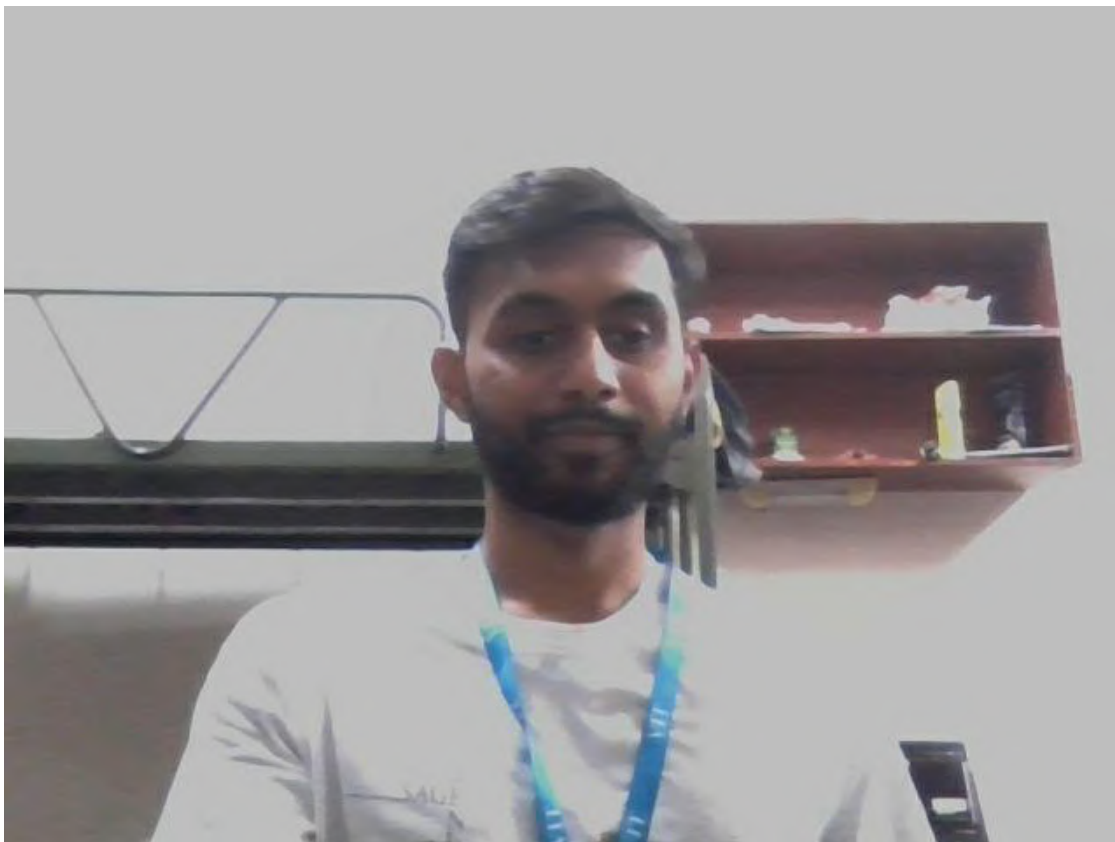
print("\nImage Shape:",frame.shape)
print("Pixel Intensity Range:",frame.min(),"to",frame.max())

```

Opening webcam... Please allow camera access.

<IPython.core.display.Javascript object>

Original Image



Grayscale Image



Canny Edge Detection Result



Image Shape: (480, 640, 3)
Pixel Intensity Range: 15 to 245

```
# =====  
# RGB Matrix Analysis  
# =====  
  
print("\nImage Shape:", frame.shape)  
  
# Split RGB channels  
B, G, R = cv2.split(frame)  
  
print("\nRed Channel Matrix (Top-left 5x5):")  
print(R[:5,:5])  
  
print("\nGreen Channel Matrix (Top-left 5x5):")  
print(G[:5,:5])  
  
print("\nBlue Channel Matrix (Top-left 5x5):")  
print(B[:5,:5])  
  
print("\nCombined RGB Matrix (Top-left 5x5):")  
print(frame[:5,:5])  
  
# =====  
# Display RGB Channels  
# =====
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(12,8))
```

```
plt.subplot(2,2,1)
plt.title("Red Channel")
plt.imshow(R, cmap='Reds')
plt.axis("off")
```

```
plt.subplot(2,2,2)
plt.title("Green Channel")
plt.imshow(G, cmap='Greens')
plt.axis("off")
```

```
plt.subplot(2,2,3)
plt.title("Blue Channel")
plt.imshow(B, cmap='Blues')
plt.axis("off")
```

```
plt.subplot(2,2,4)
plt.title("Original RGB Image")
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
plt.axis("off")
```

```
plt.show()
```

Image Shape: (480, 640, 3)

Red Channel Matrix (Top-left 5x5):

```
[[191 191 191 191 191]
 [191 191 191 191 191]
 [191 191 191 191 191]
 [191 191 191 191 191]
 [191 191 191 191 191]]
```

Green Channel Matrix (Top-left 5x5):

```
[[191 191 191 191 191]
 [191 191 191 191 191]
 [191 191 191 191 191]
 [191 191 191 191 191]
 [191 191 191 191 191]]
```

Blue Channel Matrix (Top-left 5x5):

```
[[191 191 191 191 191]
 [191 191 191 191 191]
 [191 191 191 191 191]
 [191 191 191 191 191]
 [191 191 191 191 191]]
```

Combined RGB Matrix (Top-left 5x5):

```
[[[191 191 191]
  [191 191 191]
  [191 191 191]
  [191 191 191]
  [191 191 191]]]
```

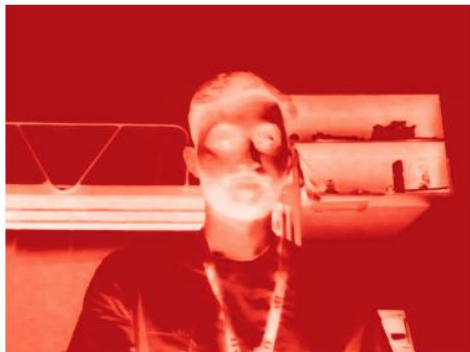
```
[[191 191 191]
 [191 191 191]
 [191 191 191]
 [191 191 191]
 [191 191 191]]
```

```
[[191 191 191]
 [191 191 191]
 [191 191 191]
 [191 191 191]
 [191 191 191]]
```

```
[[191 191 191]
 [191 191 191]
 [191 191 191]
 [191 191 191]
 [191 191 191]]
```

```
[[191 191 191]
 [191 191 191]
 [191 191 191]
 [191 191 191]
 [191 191 191]]]
```

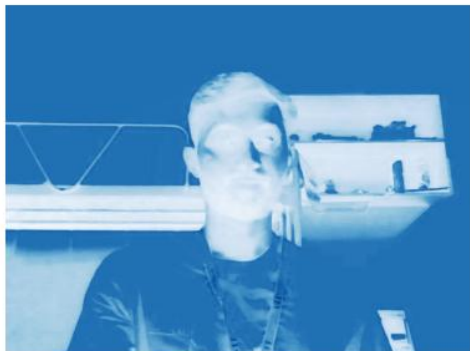
Red Channel



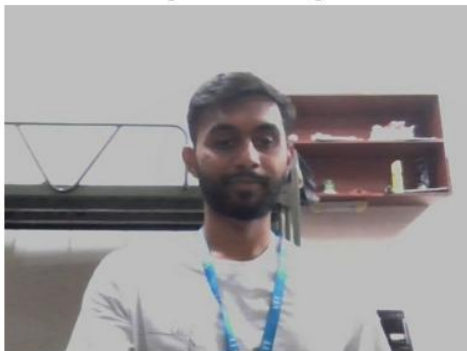
Green Channel



Blue Channel



Original RGB Image



```
# =====
# Grayscale Matrix
# =====
```

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
print("\nGrayscale Matrix (Top-left 5x5):")  
print(gray[:5,:5])
```

```
plt.figure(figsize=(5,5))  
plt.title("Grayscale Image")  
plt.imshow(gray, cmap='gray')  
plt.axis("off")  
plt.show()
```

Grayscale Matrix (Top-left 5x5):

```
[[191 191 191 191 191]  
 [191 191 191 191 191]  
 [191 191 191 191 191]  
 [191 191 191 191 191]  
 [191 191 191 191 191]]
```

Grayscale Image



Number Plate Detection

Project Description

This project focuses on developing the backend system for a number plate detection application. The main objective of the project is to build a service that can receive vehicle images, process them to detect number plates using a machine learning model, and efficiently manage the storage and retrieval of these files. The system is designed to handle image uploads, process them through a detection model, and maintain organized storage while keeping track of all file operations.

The backend of the system is developed using the Spring Boot framework, which allows the creation of scalable and well-structured REST APIs. The application follows a layered architecture where different components handle specific responsibilities, such as request handling, business logic, and database operations. This structured approach improves the maintainability and scalability of the system.

To store and manage uploaded files, an FTP server environment was configured using Windows Internet Information Services (IIS). The FTP server enables the backend application to upload, retrieve, and organize image files in remote directories. A dedicated user account was created within the Windows system to control access to the FTP server, and appropriate read and write permissions were assigned to ensure secure file handling.

The system provides multiple API endpoints to support various functionalities. One of the key endpoints allows users to upload images. When an image is uploaded, the backend automatically checks the directory structure on the FTP server and creates any missing folders before saving the file. After the image is successfully uploaded, it is passed to a machine learning model that detects the vehicle number plate and generates a processed image containing bounding boxes and labels around the detected plate.

Another API endpoint allows images to be retrieved directly from the FTP server. This feature is mainly used by the frontend to display stored images. The backend streams the image data directly to the client and sets the correct content type so that the image can be viewed properly in browsers or applications.

The system also includes a functionality for downloading entire folders as a compressed ZIP archive. This feature enables users to retrieve multiple files at once. The backend reads files directly from the FTP server and packages them into a ZIP file while streaming the data to the client, which eliminates the need to temporarily store files on the server.

To ensure proper tracking and accountability, a database auditing system is implemented using PostgreSQL. Every time a file is uploaded, the system records details such as the file name, its storage path on the FTP server, and the exact time of upload. Hibernate and Spring Data JPA are used to interact with the database, allowing the application to manage database operations through Java objects instead of manually writing SQL queries.

During the development process, certain technical challenges were encountered, particularly related to navigating directories on the FTP server and handling authentication within the Windows IIS environment. These issues were resolved by implementing improved directory management logic and properly configuring user permissions.

Overall, the project successfully implements a backend microservice that integrates REST APIs with an FTP-based storage system. By combining automated file management, machine learning-based number plate detection, and database auditing, the system provides a reliable and efficient solution for handling vehicle image processing and storage.

